

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	C. Anu Keerthana	Reg. No.:	192472057
Section / Batch:		Date:	

Code of Conduct

I C. Anu Keerthana(192472057) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C. Anu Keerthana

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries Example

user query:

“Show me the transactions with the card from last month”

- Analyze the ambiguity present in the given sentence using CFG
- Evaluate the limitations of CFG in handling real-world conversational queries
- Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing
- Justify how your proposed method improves accuracy and efficiency in an industry setting

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response Example

command:

“Book a flight to Delhi with a window seat”

- Analyze how the given sentence can lead to multiple parse structures
- Analyze the limitations of top-down parsing in this real-time application
- Analyze how Earley parsing can improve handling of ambiguity and partial inputs
- Compare and analyze the performance of top-down vs Earley parsing for such industry use cases

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

- Design a complete NLP architecture integrating:

CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

- Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).
- Create a method to:

Incorporate sub-categorization frames for medical verbs

Handle real-time data processing

Improve accuracy and scalability in a hospital environment

QUESTION 1 – Banking Customer Support Chatbot Industry Scenario

A banking company is developing an AI-powered customer support chatbot using Context-Free Grammar (CFG). However, the system has problems with ambiguous sentences, subject–verb agreement, and inefficient parsing of long queries.

Given query:

“Show me the transactions with the card from last month.”

a) Analyze the ambiguity present in the given sentence using CFG.

The sentence contains **syntactic and semantic ambiguity** because the phrase “with the card” can be attached to different parts of the sentence.

The basic sentence can be represented as:

S → **VP**

VP → **V NP**

where:

- **V = Show**
- **NP = the transactions with the card from last month**

The noun phrase can be expanded as:

NP → **Det N PP PP**

where:

- **Det = the**
- **N = transactions**
- **PP = with the card**
- **PP = from last month**

The ambiguity mainly occurs because the prepositional phrases can modify different parts of the sentence.

Interpretation 1 – Card is associated with transactions

The intended meaning could be:

Show [the transactions [with the card] [from last month]].

Here:

- “with the card” specifies which transactions are being requested.
- “from last month” specifies the time period.

Semantic representation:

SHOW(Transactions, Card, LastMonth)

Meaning:

Show the transactions that were made using the card during last month.

Interpretation 2 – “with the card” modifies the action

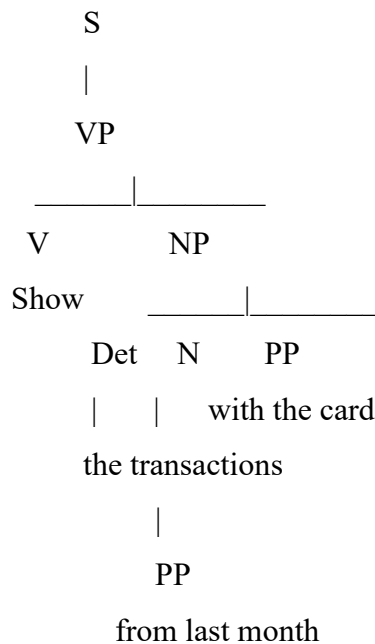
Another possible interpretation is:

Show [the transactions] [with the card] [from last month].

Here, “with the card” can be interpreted as a separate instruction concerning how the transaction information should be accessed or displayed.

This interpretation is less likely in the banking context but is syntactically possible.

Parse representation



The ambiguity arises because CFG primarily represents **syntactic structure** and does not inherently determine which interpretation is semantically intended.

b) Evaluate the limitations of CFG in handling real-world conversational queries.

CFG is useful for representing grammatical structures, but it has several limitations in conversational banking applications.

1. Difficulty handling ambiguity

CFG can generate multiple valid parse trees without determining which one matches the user's intended meaning.

For the given query, the system may generate different structures for the prepositional phrases.

2. Lack of semantic information

CFG focuses mainly on syntax.

For example, it can identify:

transactions → noun

but it does not inherently understand that:

transactions + card + last month

represents a banking transaction query.

3. Subject–verb agreement

Basic CFG rules may accept grammatically incorrect combinations unless separate agreement rules are explicitly defined.

For example:

“The customer check the balance.”

A basic CFG may require additional mechanisms to reject this sentence.

4. Poor handling of long sentences

Long conversational queries can generate a large number of possible parse trees. This increases computational cost and can cause inefficient parsing.

5. Limited conversational context

Banking users may use incomplete expressions such as:

“Show last month's ones.”

A pure CFG may struggle because the intended noun, such as “transactions,” is omitted.

6. Domain-specific language

Customers may use informal expressions:

“How much did I spend last month?”

“Show card spends.”

A grammar designed only for formal sentences may fail to interpret these variations.

Therefore, CFG alone is insufficient for a robust banking chatbot.

c) Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing.

A better solution is to use a hybrid parsing architecture consisting of:

PCFG + Feature Structures + Earley Parsing + Semantic Analysis

1. PCFG – Probabilistic Context-Free Grammar

PCFG assigns probabilities to grammar rules.

For example:

VP $\rightarrow$ V NP [0.70]

NP $\rightarrow$ Det N PP [0.30]

When multiple parse trees are possible, the system can choose the tree with the highest probability.

For the banking query, the parser can learn from historical banking queries that:

“transactions with the card”

is more likely to refer to transactions performed using a particular card.

Therefore, PCFG helps resolve ambiguity based on learned probabilities.

2. Feature Structures

Feature structures can represent grammatical properties such as:

- Number
- Person
- Tense
- Agreement

For example:

NP:

Number = Singular

VP:

Number = Singular

The system can reject mismatched structures such as:

“The customer check the balance.”

and prefer:

“The customer checks the balance.”

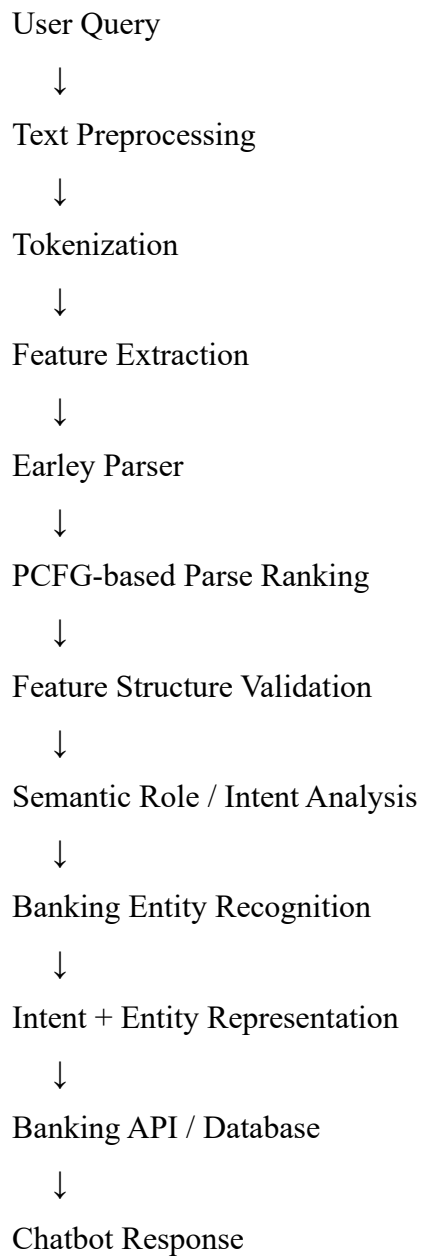
3. Earley Parsing

Earley parsing is useful because it can efficiently handle:

- Ambiguous grammars
- Complex sentences
- Partial inputs
- Different possible parse structures

It is especially useful when customers produce long or incomplete queries.

Proposed architecture



d) Justify how the proposed method improves accuracy and efficiency in an industry setting.

The proposed hybrid approach provides several advantages.

Problem	CFG	Proposed Solution
Ambiguity	Generates multiple parses	PCFG ranks likely interpretations
Agreement	Limited	Feature structures enforce agreement
Long queries	Can become inefficient	Earley parsing handles complex/ambiguous input
Informal queries	Limited	Semantic and contextual processing improves interpretation

Banking entities	Not inherently represented	Entity recognition identifies cards, transactions and dates
Real-time response	May be inefficient for complex inputs	Efficient parsing and ranking reduce unnecessary processing

For example:

Input:

“Show me the transactions with the card from last month.”

The proposed system extracts:

Intent: SHOW_TRANSACTIONS

Entity: CARD

Time: LAST_MONTH

Action: RETRIEVE

This structured representation can then be sent to the banking transaction service.

Conclusion

CFG provides the basic syntactic foundation, but it is insufficient for real-world banking conversations because it cannot effectively resolve ambiguity or understand contextual meaning. A hybrid approach using Earley parsing, PCFG, feature structures, and semantic analysis can improve interpretation accuracy, agreement handling, and parsing efficiency. This makes the system more suitable for an industry-grade banking chatbot.

QUESTION 2 – Real-Time Voice Assistant

Industry Scenario

A voice assistant similar to Alexa or Siri processes spoken commands using top-down parsing. The system experiences backtracking, difficulty with incomplete or ambiguous input, and slow real-time responses.

Given command:

“Book a flight to Delhi with a window seat.”

a) Analyze how the given sentence can lead to multiple parse structures.

The sentence contains the main components:

- **Book** → Verb
- **a flight** → Object
- **to Delhi** → Destination
- **with a window seat** → Seat preference

The intended interpretation is:

Book a flight whose destination is Delhi and whose seat preference is a window seat.

A semantic representation can be:

BOOK(Flight)

with:

Destination = Delhi

SeatPreference = Window

Possible structural interpretation

S

|

VP

|_____

V

NP

|

|

Book

a flight

|

PP

PP

|

|

to Delhi

with a window seat

The first interpretation is:

BOOK(Flight, Destination=Delhi, Seat=Window)

However, the phrase “**with a window seat**” can potentially be attached differently depending on the grammar.

The parser may temporarily consider:

[Book [a flight to Delhi with a window seat]]

or alternative attachment structures in which the PP is associated with another constituent.

Although the intended meaning is clear to a human, a parser may have to evaluate multiple structural possibilities.

b) Analyze the limitations of top-down parsing in this real-time application.

Top-down parsing begins from the start symbol and predicts grammar rules until it reaches the input.

For example:

S

↓

VP

↓

V NP

↓

Book NP

However, this approach has several limitations.

1. Backtracking

If an early prediction is incorrect, the parser must return to an earlier state and try another rule.

This increases computation.

2. Ambiguity

Multiple possible structures may cause the parser to explore several alternatives.

3. Left-recursive grammar problems

Traditional top-down parsers can have difficulty with left-recursive rules, potentially causing repeated expansion.

4. Incomplete spoken input

Voice assistants frequently receive incomplete commands such as:

“Book a flight to...”

The system must be able to interpret partial input and wait for additional information.

5. Real-time requirements

Voice assistants require fast response times. Excessive backtracking can increase latency.

6. Speech recognition errors

Spoken input may contain errors such as:

“Book a flight to Delhi with a window...”

The parser must tolerate incomplete or noisy input.

Therefore, a basic top-down parser is not ideal for real-time voice assistants.

c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs.

Earley parsing is particularly suitable for this scenario because it supports general CFG parsing and can handle ambiguous and incomplete structures.

Earley parsing uses three major operations:

1. Prediction

The parser predicts possible grammar rules.

2. Scanning

It matches predicted grammar symbols with the input tokens.

3. Completion

It completes grammar rules when their required constituents have been recognized.

For:

“Book a flight to Delhi with a window seat.”

Earley parsing can maintain multiple possible states simultaneously instead of repeatedly restarting the parsing process.

Example

Input

↓

Book

↓

a flight

↓

to Delhi

↓

with a window seat

↓

Earley State Chart

↓

Possible Parse Structures

↓

Semantic Ranking



Final Interpretation

It can also process partial input:

“Book a flight to Delhi with...”

and maintain possible interpretations until the next words arrive.

This is valuable for conversational voice systems.

d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases.

Feature	Top-Down Parsing	Earley Parsing
Parsing approach	Starts from grammar start symbol	Uses dynamic chart parsing
Ambiguity	Can require backtracking	Can maintain multiple possibilities
Partial input	Limited	Better supported
Backtracking	High in ambiguous cases	Reduced through chart states
General CFG support	Depends on implementation	Strong
Real-time suitability	Limited for complex ambiguity	Better for complex inputs
Memory usage	Can be lower for simple grammar	Uses chart/state storage
Complex sentences	Can become inefficient	Handles them systematically
Voice assistant suitability	Moderate	High

Industry evaluation

For a simple command:

“Call Mom.”

top-down parsing may be sufficiently fast.

However, for commands containing multiple entities and modifiers:

“Book a flight to Delhi with a window seat for tomorrow morning.”

Earley parsing becomes more useful because it can manage multiple grammatical possibilities and partial input.

Conclusion

Top-down parsing is simple and effective for small, unambiguous grammars but suffers from backtracking and ambiguity in conversational applications. Earley parsing provides a more flexible solution for complex, ambiguous, and incomplete spoken commands. Therefore, Earley parsing is more appropriate for a real-time voice assistant handling diverse natural-language commands.

QUESTION 3 – Healthcare Medical Report Analysis

Industry Scenario

A healthcare company is developing an AI system that extracts information such as diagnosis, prescribed treatment, and follow-up actions from medical reports.

The system must handle:

- Complex and lengthy sentences
- Structural ambiguity
- Subject–verb agreement
- Medical action–entity relationships
- Real-time processing

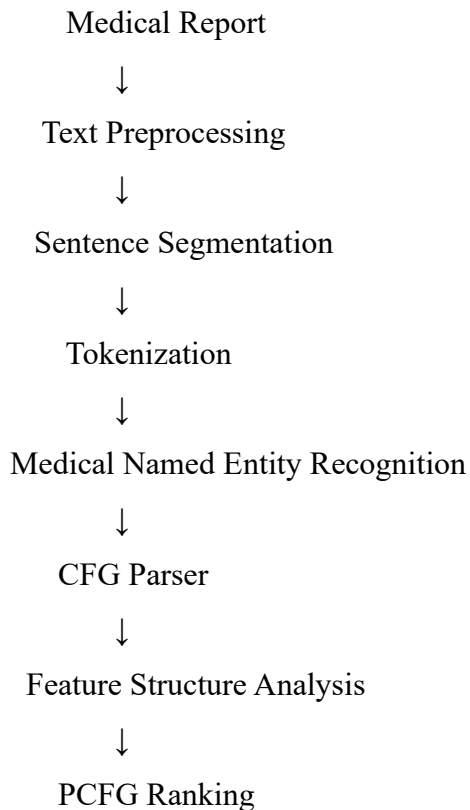
Given sentence:

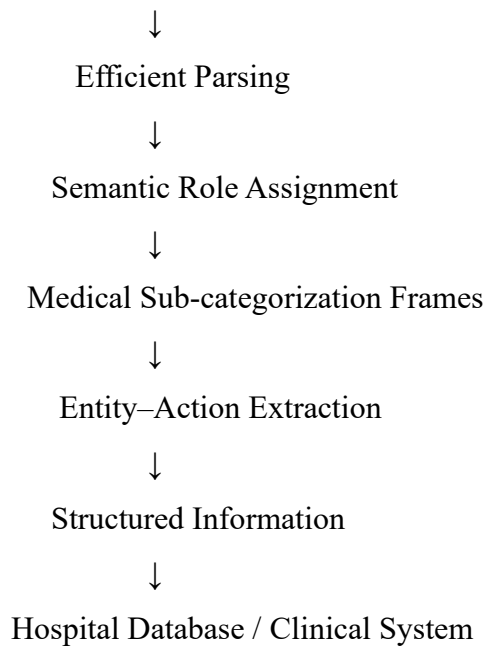
“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

a) Design a complete NLP architecture integrating CFG, PCFG, Feature Structures and efficient parsing techniques.

A robust healthcare NLP architecture should combine syntactic parsing, probabilistic disambiguation, feature structures, semantic analysis, and domain-specific knowledge.

Proposed Architecture





Module 1 – Preprocessing

The medical report is divided into sentences and tokens.

Input:

The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.

Tokens include:

The / doctor / who / reviewed / the / patient / last / week / recommends / starting / medication / and / scheduling / a / follow-up / visit / in / Chennai

Module 2 – CFG

CFG identifies the basic syntactic structure.

A simplified grammar can be:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$NP \rightarrow NP RelativeClause$

$VP \rightarrow V NP$

$VP \rightarrow V GerundPhrase$

$VP \rightarrow V GerundPhrase Conjunction GerundPhrase$

$RelativeClause \rightarrow RelPronoun VP$

PP → P NP

This enables the system to identify the relationship between the doctor, patient, medication, follow-up visit, and location.

Module 3 – Feature Structures

Feature structures can encode grammatical properties.

For example:

Doctor:

Number = Singular

Role = Agent

Recommends:

Tense = Present

Number = Singular

Subject = Doctor

Because **doctor** is singular, the verb **recommends** is correctly matched to the subject.

Module 4 – PCFG

When several parses are possible, PCFG assigns probabilities to competing parse structures.

The most probable interpretation is selected based on learned medical-language patterns.

Module 5 – Efficient Parsing

For complex medical sentences, efficient parsing techniques such as **Earley parsing** can maintain partial parse states and handle ambiguity without repeatedly starting from the beginning.

Module 6 – Semantic Analysis

The parser extracts:

- Agent
- Action
- Patient
- Treatment
- Follow-up
- Location
- Time

Module 7 – Structured Output

The final information can be represented as:

Agent: Doctor

Related Patient: Patient

Previous Action: Reviewed

Time: Last week

Recommended Action 1: Start medication

Recommended Action 2: Schedule follow-up visit

Location: Chennai

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output.

Step 1 – Input

The system receives:

“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

Step 2 – Tokenization

The sentence is separated into individual tokens.

The | doctor | who | reviewed | the | patient |
last | week | recommends | starting | medication |
and | scheduling | a | follow-up | visit | in | Chennai

Step 3 – Syntactic Analysis

CFG identifies:

Main Subject = The doctor

Relative Clause = who reviewed the patient last week

Main Verb = recommends

Action 1 = starting medication

Action 2 = scheduling a follow-up visit

Location = Chennai

Step 4 – Feature Structure Analysis

The system verifies:

Subject:

doctor

Number = Singular

Verb:

recommends

Number = Singular

Tense = Present

Thus:

doctor → recommends

is grammatically consistent.

Step 5 – PCFG Disambiguation

The system evaluates possible parse structures and selects the most probable interpretation based on medical-language patterns.

Step 6 – Semantic Role Extraction

The system identifies:

Entity	Semantic Role
Doctor	Agent
Patient	Patient/Entity being reviewed
Medication	Treatment
Follow-up visit	Follow-up Action
Chennai	Location
Last week	Time

Step 7 – Sub-categorization Analysis

The verb **recommend** requires an action or proposition as its complement.

Here:

recommends → starting medication

and

recommends → scheduling a follow-up visit

Therefore, both actions are associated with the doctor's recommendation.

Step 8 – Structured Output

The extracted information can be represented as:

{

Agent: Doctor,

```
Patient: Patient,  
PreviousAction: Reviewed,  
Time: Last Week,  
RecommendedTreatment: Starting Medication,  
FollowUpAction: Scheduling Follow-up Visit,  
Location: Chennai  
}
```

c) Create a method to incorporate sub-categorization frames for medical verbs, handle real-time data processing, and improve accuracy and scalability.

1. Sub-categorization frames for medical verbs

Sub-categorization describes the types of complements that a verb can accept.

For example:

Recommend

recommend + Gerund Phrase

recommend + Noun Phrase

recommend + that-clause

In the given sentence:

recommend + starting medication

and

recommend + scheduling a follow-up visit

are valid structures.

Review

review + Patient/Object

Example:

Doctor reviewed patient.

Schedule

schedule + Event

Example:

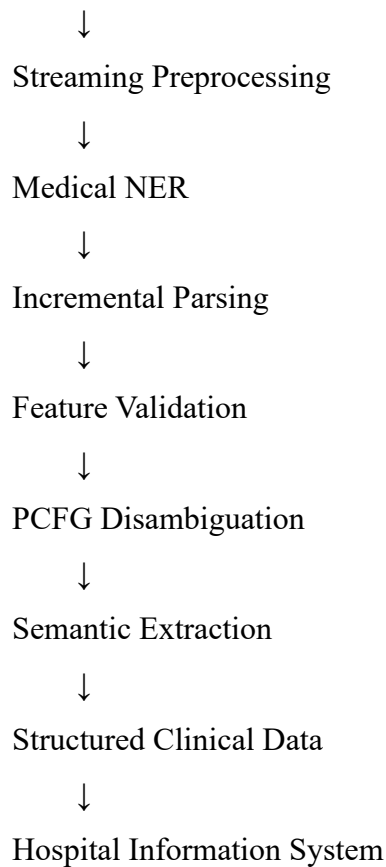
schedule a follow-up visit

These frames help the NLP system understand the relationship between medical actions and entities.

Real-Time Processing Method

For hospital environments, the system should use a pipeline optimized for low latency.

Incoming Medical Report



Techniques for real-time performance

1. **Incremental parsing** – process text as it arrives instead of waiting for the complete document.
2. **Caching** – reuse frequently used grammar and medical terminology.
3. **Efficient parsing** – use chart-based parsing to avoid unnecessary recomputation.
4. **Parallel processing** – process independent sections simultaneously.
5. **Domain-specific models** – use medical NLP models to reduce unnecessary computation.
6. **Confidence scoring** – automatically identify uncertain extractions for review.
7. **Database indexing** – provide fast access to medical terminology and relationships.

Accuracy Improvement

Accuracy can be improved using a combination of:

CFG

Provides the fundamental syntactic structure.

PCFG

Resolves ambiguity using probabilities learned from medical data.

Feature Structures

Ensure grammatical agreement such as:

doctor → recommends

rather than an incorrect form such as:

doctor → recommend

Medical NER

Identifies:

- Diseases
- Medicines
- Symptoms
- Doctors
- Patients
- Locations
- Procedures

Semantic Role Labeling

Identifies relationships such as:

Doctor → Agent

Patient → Patient/Entity

Medication → Treatment

Chennai → Location

Medical Ontologies

A medical knowledge base can help connect terms such as:

medication → treatment

and

follow-up visit → clinical follow-up action

Scalability in a Hospital Environment

The system should be designed to process large volumes of medical reports.

A scalable architecture can use:

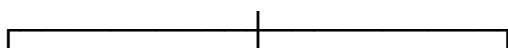
Multiple Hospital Data Sources

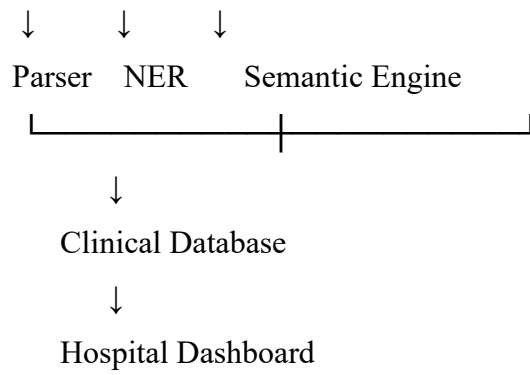


API / Data Gateway



Distributed NLP Pipeline





The system can scale by adding processing resources when the number of reports increases.

For safety-critical information, the system should also maintain:

- Confidence scores
- Audit logs
- Validation rules
- Human review for uncertain results

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge

Course Coordinator

Program Director

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____